

Task-Spec

CONVERGE PASS 5 · THE CORNERSTONE UNIT · V3.6.0 — SIX TIERS, DUAL GATES, ONE SEALED FILE

The Unit That Verifies *Itself*.

Every AI coding tool consumes some form of "task." Most of those are PRDs — written for humans, interpreted by humans, judged by humans. That makes agents *non-autonomous*: they need a person in the middle of the loop. Task-Spec fixes that. Each spec carries its own **runnable bash success criteria**, so an agent reads it, executes, runs the evals, and loops on failure without a human in the middle. This is **Eval-Driven Development** — the same conceptual leap as TDD or Infrastructure-as-Code.

6	6	2	3	HMAC
TIERS · XS → XXL LEAVES & NODES	ZONES PER SPEC	GATES · DUALS PRE & POST	CONFORMANCE LEVELS L0-L2	SEALED · THE ONLY INSTRUCTION SOURCE

DOCUMENT	FORMAT VERSION
The cornerstone unit, in depth · v3.6.0	<code>format_version: 3</code> · additive since 3.0

One format, many executors — Claude, Codex, Kimi, Gemini, taskship, anthive, or a human with a terminal. The eval, not the agent, defines done, which is what makes the coding engine a commodity slot. **No eval, not a task yet.**

Why the unit had to change

FROM "A DOCUMENT A HUMAN JUDGES" TO "A FILE THAT JUDGES ITSELF"

A PRD is a *request for interpretation*. It describes an outcome in prose and leaves the question of whether the outcome was achieved to a reviewer. That is a perfectly good contract between two humans — and a broken one between a human and a machine, because the machine cannot execute a standard it cannot read, and cannot know when to stop.

Task-Spec inverts the contract. The definition of done **travels inside the file**, as bash. That single move changes four things at once:

DIMENSION	PROSE TASK (PRD)	TASK-SPEC (EDD)
"Done" is decided by	human review	bash evals returning 0
Iteration cadence	hours – days	seconds – minutes
Portability	vendor-specific	universal — markdown + YAML + bash
Audit trail	narrative	machine ledger
Catches ambiguity	maybe	yes — the eval fails to be writable
Authoring cost	faster (prose)	slower — evals are harder to write

THE HONEST COST

That last row is real and is not hidden. Writing a runnable eval is harder than writing a paragraph, and the difficulty is the point: **if you cannot express done as a check, you have not finished thinking**. The cost is paid once, by the author, instead of every time, by a reviewer.

THE RULE THAT MAKES THE WHOLE METHOD WORK

No eval, not a task yet. A spec without a runnable check is a wish. Because the eval — and not the agent — defines done, any coding agent becomes a swappable commodity slot: swap Claude for Codex for Kimi and the gate is unmoved.

Anatomy — frontmatter plus six zones

EVERYTHING THE EXECUTOR NEEDS, AND NOTHING IT MUST GUESS

ZONE	NAME	WHAT IT HOLDS · WHY
1	Intent	Why this task exists — goal + bounded context. Keep it short; a Context over ~100 lines means you wrote a PRD.
2	Behavior	GIVEN / WHEN / THEN statements, each labelled B-N . Every behavior demands at least one eval — this is the traceability spine.
3	Contract THE MOAT	Runnable bash eval_N() functions + the Validation Card (YAML) + the Exit Check. This is the self-verifying core.
4	Guardrails	Anti-patterns and do-not-touch paths — bound the agent's blast radius, specifically. "Be careful" is not a guardrail.
5	Operations	Open questions — admit unknowns honestly so a stalled loop knows where to ask.
6	Rollback + Observability full profile only	How to reverse a failed execution, and what to watch during and after it.

PROFILES — ZONES SCALE WITH EFFORT

Not every task earns every zone. A `minimal` spec carries the load-bearing zones only; the `full` profile adds Rollback + Observability. The profile is effort-scaled, so an XS config bump does not have to carry a rollback plan it will never use.

Frontmatter — the machine-readable header

```
---
id: T-20260519-add-health-endpoint      # T-YYYYMMDD-<kebab-slug>
status: ready                           # ready|in-progress|blocked|done|parked
format_version: 3
profile: standard                        # minimal | standard | full
effort: S                               # XS·S·M·L leaf | XL·XXL node
budget_iterations: 10                   # the retry ceiling — not a success criterion
execution_backend: any                  # OPEN STRING — the backend names itself
depends_on: []                           # the DAG edges — cycle-checked
touches_paths: [src/api/server.py, tests/test_health.py] # blast radius
tracker_ref: (none)                     # receipt written BACK by Pass 6 — never by hand
signed_off: false                       # only the PRE gate flips this
accepted: false                         # only the POST gate flips this
```

TWO FIELDS NO HUMAN MAY WRITE

`signed_off` and `accepted` are gate outputs, not author inputs. Hand-stamping either is rejected — the sign-off envelope check recomputes an HMAC over a canonical payload and compares. `tracker_ref` is likewise a receipt, written back by Register after a successful upsert.

A complete spec, end to end

~95 LINES · EVERY BEHAVIOR EVAL-COVERED · EVERY EVAL TRACED TO A BEHAVIOR

```
## Behavior
- B-1 — GIVEN the api server is running WHEN a client GETs /health
      THEN it responds HTTP 200 with body {"status":"ok"}
- B-2 — GIVEN the new endpoint exists WHEN the test suite runs
      THEN a regression test for /health passes

## Success Criteria          # THE MOAT — runnable, deterministic
```bash
eval_1() { uvicorn src.api.server:app --port 8001 & PID=$!; sleep 2; kill -0 $PID 2>/dev/null; }
eval_2() { curl -fs http://localhost:8001/health | jq -e '.status = "ok"'; }
eval_3() { pytest tests/test_health.py -q; }
```

## Validation Card          # the machine-readable index of the evals
```yaml
success_criteria:
 - {id: eval_1, verifies: [B-1], check_type: deterministic, terminal: true}
 - {id: eval_2, verifies: [B-1], check_type: deterministic, terminal: true}
 - {id: eval_3, verifies: [B-2], check_type: deterministic, terminal: true}

retry_policy:
 max_iterations: 10
 circuit_breaker_no_progress: 3
 on_terminal_failure: park_with_context

agent_contract:
 version: 2
 required_tools: [git, bash, python3]
 emit: [pass, fail, retry_with_reason, parked_with_context]
```

## Exit Check
```bash
eval_1 && eval_2 && eval_3
```
```

TRACEABILITY

Both directions

B-1 is covered by `eval_1 + eval_2`; B-2 by `eval_3`. Every behavior has an eval and every eval names a behavior — an untraced behavior FAILS the Definition of Done.

THE CIRCUIT BREAKER

Stop patching — now enforced

`circuit_breaker_no_progress: 3` ends the "try the same thing louder" failure mode; on terminal failure the task **parks with context** rather than looping or silently dying. Since cvg 0.20.0 the Pass 8 loop kernel **enforces** these fields — a stalled run lands `STALLED`, an exhausted budget lands `EXHAUSTED` with a handoff — where before they were declared and nothing ran them.

The six-tier engine — leaves run, nodes decompose

V3.4 MADE THE CUT RECURSIVE · SIZE BY CONTEXT, VERIFICATION, AND BLAST RADIUS

Sizing is **agent-native**: not human hours, but how much context the unit needs, how it verifies, and how wide it writes. Two kinds. **Leaves** (XS/S/M/L) are directly runnable atoms that fit one fresh executor session. **Nodes** (XL/XXL) are decomposition directives — not runnable at all; they **MUST** declare a `children:` block and expand into leaves.

| SIZE | KIND | WRITE-SURFACE BUDGET | EXAMPLE |
|------|--------------------|---------------------------------------|---|
| XS | leaf | ≤ 1 path | a config key; bump a dependency |
| S | leaf | ≤ 2 paths | a <code>/health</code> endpoint + its test |
| M | leaf | ≤ 3 paths | a dbt model + schema + test |
| L | leaf · the ceiling | ≤ 5 paths · long-horizon backend | a service slice as ONE coherent goal |
| XL | node · decomposes | children ≥ 2 · owns none | a whole swimlane $\rightarrow$ its leg leaves |
| XXL | node · decomposes | children ≥ 3 · owns none | a whole backbone $\rightarrow$ swimlane nodes |

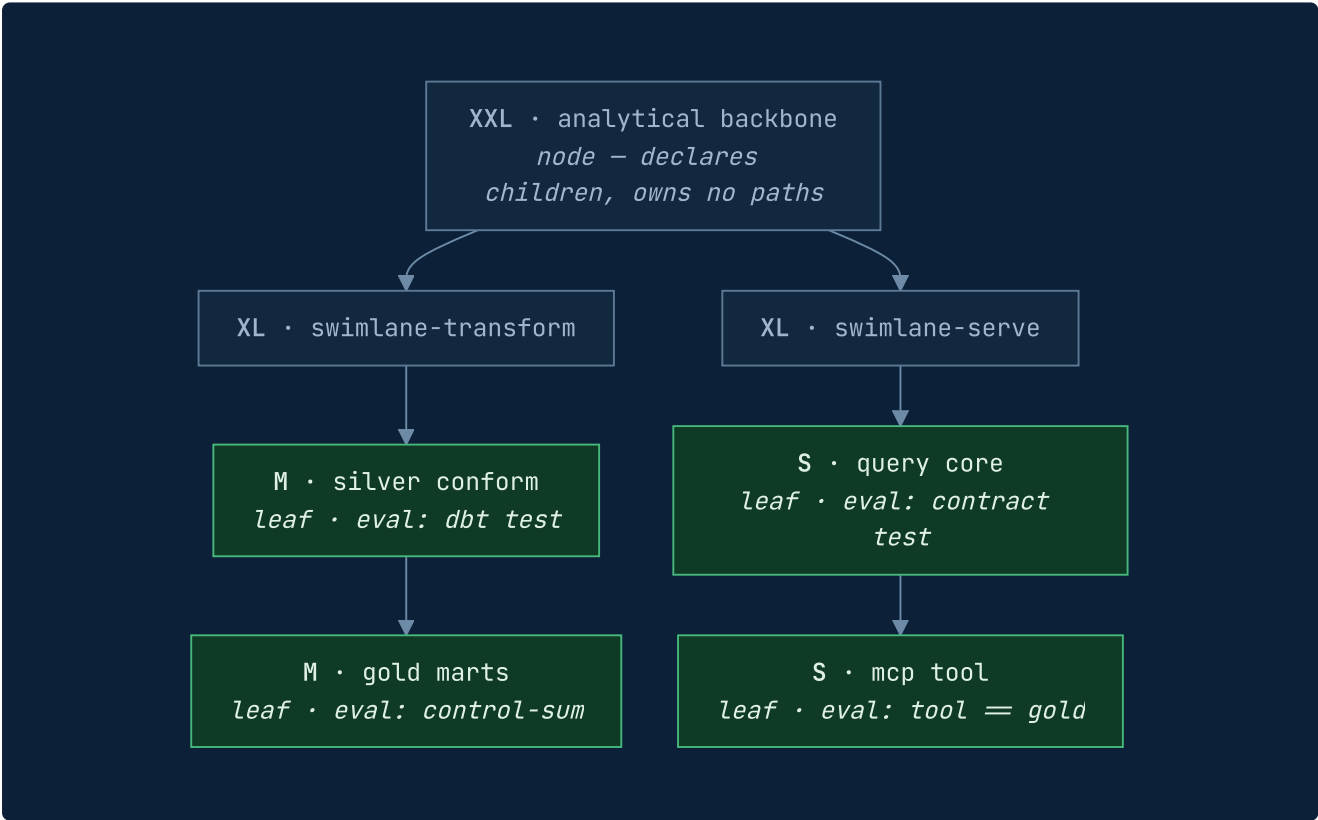


FIG 1 Descend by decomposition, run only leaves, compose verified results back up. A breached write-surface budget is a *signal that the decomposition was too coarse* — split further.

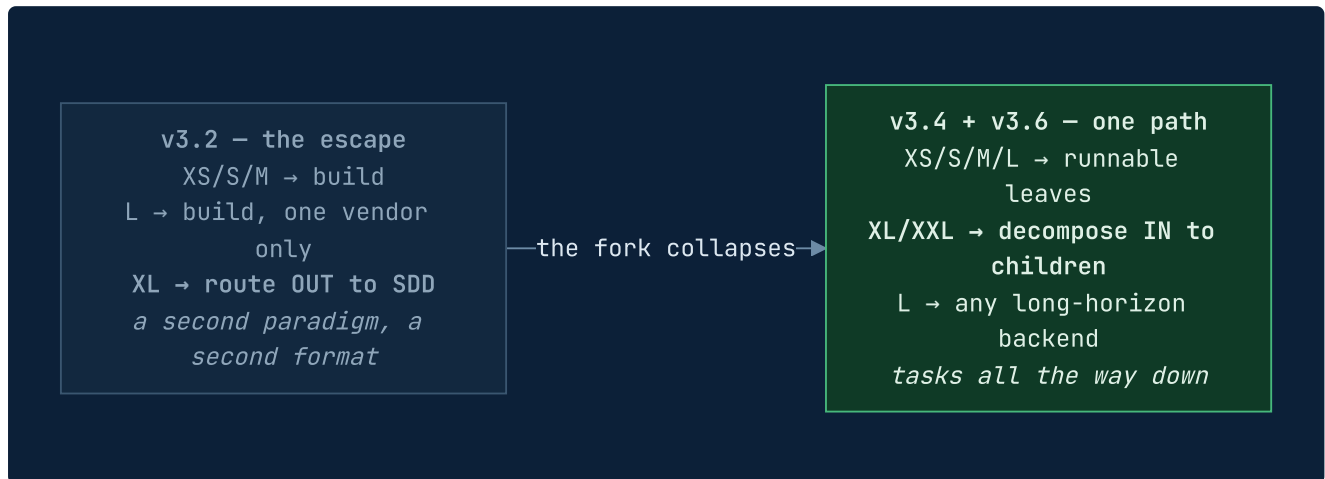
THE GATE IS ENFORCEMENT, NOT ADVICE

A leaf over its budget **WARNs** *"mis-sized — split or reclassify UP."* An XL/XXL node with too few children **FAILs** *"must decompose — no route out."* And `safe-to-delegate` **refuses to delegate a node at all** — a worker dispatches a node's children, never the node itself.

Recursion, not escalation

V3.4 REMOVED THE SDD ESCAPE · V3.6 REMOVED THE VENDOR LOCK

Task-Spec used to have an exit. The old effort gate routed `XL → SDD`: work above a size threshold left the format entirely for a plan-driven paradigm (AgentSpec / OpenSpec / Spec Kit). v3.4 removed that route. Big work now decomposes **in** — the same format, self-similar at every scale — rather than escalating **out**.



The argument for the change is empirical: frontier engines execute well-scoped atoms reliably, and a *tree of verified atoms composes back up more safely than one monolithic spec*. A monolith hides which step regressed and makes per-stage evals unenforceable. Decomposition keeps every defect behind its own green check.

v3.6 — the L tier stops naming a vendor

Until v3.6, `effort: L` hard-failed unless `execution_backend: glm`. That contradicted the format's own rule that `execution_backend` is an **open string** — "the backend names itself; the values are non-normative examples, not an allow-list." The gate's real intent was *an L leaf needs a long-horizon builder*, not *an L leaf needs one specific vendor*.

```
# the gate now checks membership, not identity
TS_LONG_HORIZON_BACKENDS="${TASKSPEC_LONG_HORIZON_BACKENDS:-glm claude codex kimi}"

# override per fleet — your harness names itself
export TASKSPEC_LONG_HORIZON_BACKENDS="claude codex kimi glm my-harness"
```

WHY THIS MATTERS BEYOND ONE FLAG

It was surfaced by a real user whose fleet is Claude Code / Codex / Kimi: an L-sized spec could not name the engine that was actually going to run it. A format that claims vendor-neutrality has to mean it at the gate, not just in the prose — otherwise the neutrality is decorative.

The two gates are duals

SAME EVALS · OPPOSITE EXPECTATIONS · ONE CERTIFIES THE SPEC, THE OTHER THE WORK

This is the most misunderstood part of the format. There are **two** gates, they run the same evals, and they expect **opposite results**. The PRE gate certifies that the *spec* is safe to hand to a machine. The POST gate certifies that the *work* actually satisfied it.

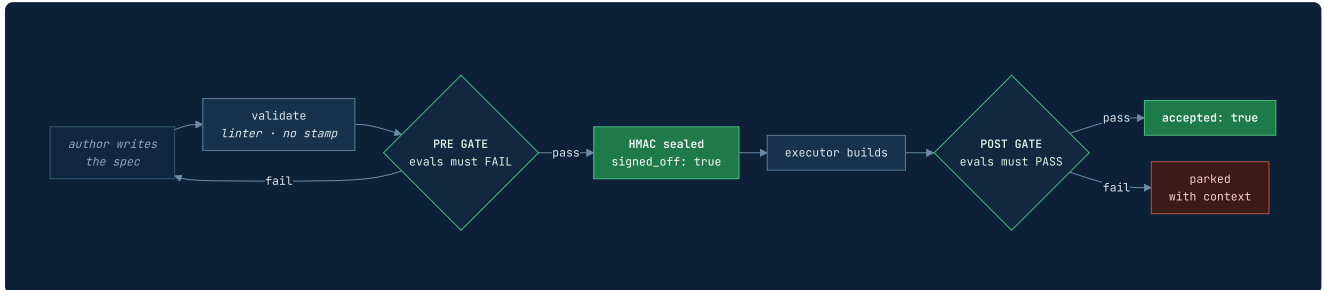


FIG 2 Same evals, opposite expectations. An eval that passes *before* any work exists is a broken eval; an eval that fails *after* it means the work is not done.

| GATE | EXPECTS THE EVALS TO | WHAT IT CERTIFIES |
|--|--------------------------------------|--|
| PRE
<code>safe-to-delegate.sh --stamp</code> | FAIL
nothing is built yet | The spec : evals are well-formed bash, the sizing gate holds, a broken-eval guard passes, and an XL/XXL node is refused outright . Then it seals the eval bodies in an HMAC envelope and writes <code>signed_off: true</code> . |
| POST
<code>accept-task.sh -stamp</code> | PASS
from a clean checkout | The work : evals re-run in a clean tree, the change set stayed inside the blast radius, the HMAC is intact, and optionally <code>--gold-sanity</code> proves the eval fails on the unpatched baseline. Writes <code>accepted: true</code> . |

VERDICT: DELEGATE

A spec is dispatchable when, and only when, the PRE gate returns **VERDICT: DELEGATE**. There is no other path — hand-editing `signed_off` is caught by the envelope check.

Anti-reward-hacking — why a green eval means something

THE ACCEPTANCE GATE ASSUMES THE EXECUTOR MAY BE WRONG, OR OPTIMISING FOR THE WRONG THING

An eval-defined "done" is only as trustworthy as its resistance to being gamed. A sufficiently motivated optimiser can satisfy a check without satisfying the intent: weaken the assertion, widen the blast radius, or write a test that would have passed anyway. The POST gate is built on the assumption that this will happen, and each layer closes one route.

| LAYER | WHAT IT CLOSES |
|----------------------------|---|
| Re-run, never trust | The gate runs the evals itself , from a clean checkout — it never accepts the executor's report of its own success. A non-zero exit is rejected automatically. |
| Blast-radius check | Every changed file must be inside <code>touches_paths</code> <code>u</code> <code>creates_paths</code> . "I fixed it by editing the test harness" is caught here. |
| HMAC contract integrity | The envelope seals the eval bodies at sign-off. If the evals were weakened <i>after</i> the spec was signed, the recomputed signature no longer matches and acceptance fails. |
| <code>--gold-sanity</code> | The Goodhart guard: the eval must FAIL on the unpatched baseline . An eval that passes without the change proves nothing — it was never discriminating. |
| Atomic state machine | Advisory lock, validated transition, tmp-file + atomic move. Two executors cannot both claim one ready task. |
| Append-only ledger | <code>_metrics.jsonl</code> is forensic and append-only; <code>_state.yaml</code> is a <i>derived</i> , <i>rebuildable</i> index. The frontmatter is the truth, so the index cannot silently diverge. |

THE EVALS MUST DISCRIMINATE

This principle governs the format's own test suites too: every gate ships with regression fixtures that prove the **negatives fail**. A fixture that wrongly **PASSES** on a previous version is treated as a defect in the gate. A check that cannot fail is not a check.

Conformance — what "this engine can run it" means

THREE CUMULATIVE LEVELS · CERTIFIED BY A REAL SUITE, NOT A CLAIM

"Vendor-portable" is a testable property, not a marketing line. An executor earns a level by driving reference specs to a defined outcome — the suite ships in the skill, and `scripts/ref-executor.sh` is an L2-conformant reference implementation.

| LEVEL | WHAT IT ADDS | HOW IT'S CERTIFIED |
|-------|---|--|
| L0 | Read the format and run the evals — given a signed spec, produce work that makes the Exit Check return 0. | Drives a solvable reference spec; the runner exits 0. |
| L1 | L0 + honor the lifecycle — acquire the lock (<code>status</code> → <code>in-progress</code>) before working, release it (→ <code>done</code>) on success. | After success the spec's <code>status</code> is <code>done</code> . |
| L2 | L1 + honor the retry budget — on a spec it cannot satisfy within <code>budget_iterations</code> , stop and park with a reason rather than loop forever. | An <i>unsolvable</i> reference spec ends <code>parked</code> , and the executor does not time out. |

L0

Supervised

safe for one-shot dispatch with a human watching.

L1

Shared backlog

the lock prevents two executors double-picking the same ready task.

L2

Unattended

a runaway task self-terminates into `parked` instead of burning budget forever.

```
# certify your own executor against the suite
bash scripts/conformance-check.sh --level L2 --executor "my-runner --flags"
```

A PLAIN BASH LOOP REACHES L0

That is deliberate — the floor is low enough that the format is genuinely portable. **L2 is the bar for an executor you would trust to run unattended**, which is the bar the dark factory actually needs.

Lifecycle and the five-layer backlog

EVERY STATE CHANGE ATOMIC, LOGGED, AND RECOVERABLE

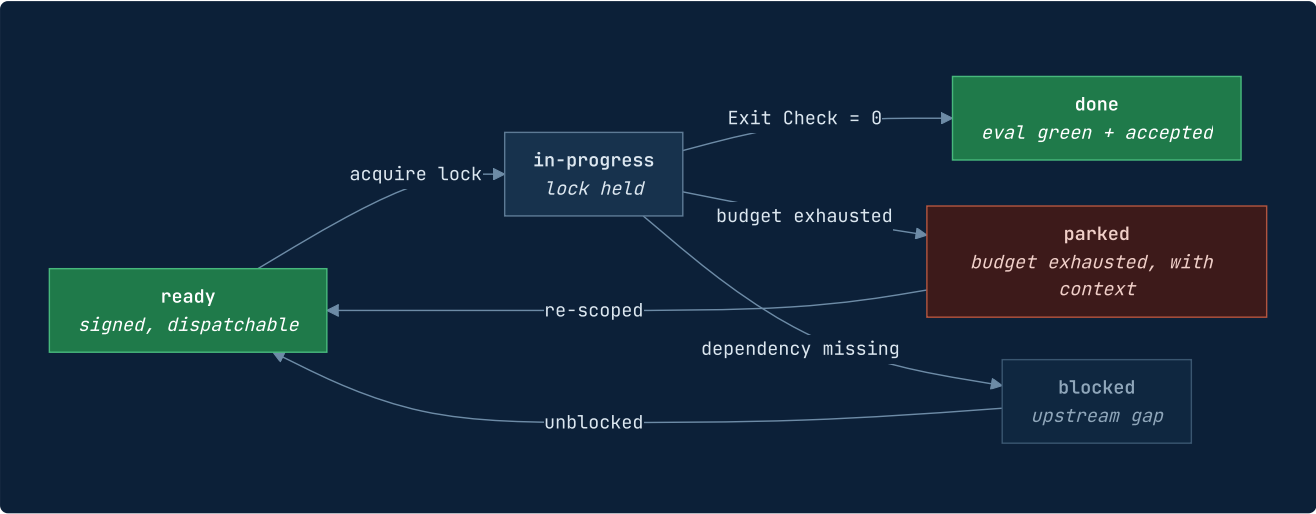


FIG 3 Transitions are validated against a real state machine and taken under an advisory lock — never by editing frontmatter directly.

| LAYER | SURFACE | ROLE |
|-------|--------------------|--|
| 1 | git | the permanent record |
| 2 | tasks/ folder | the backlog — active, done/ , parked/ |
| 3 | frontmatter status | authoritative per-file truth |
| 4 | _state.yaml | a derived index — rebuildable, never authoritative |
| 5 | _metrics.jsonl | append-only forensic ledger |

THE ANTI-PATTERN THAT BREAKS ALL OF IT

Editing `T-*.md` frontmatter by hand bypasses the lock *and* the ledger. Use `transition-status.sh`. Derived state can always be rebuilt with `rebuild-state.sh` ; a corrupted *authoritative* state cannot.

One format, many executors

THE AGENT CONTRACT IS WHAT MAKES THE CODING ENGINE A COMMODITY SLOT

Any agent consuming a Task-Spec honors the same contract. That contract — not a plugin, not an API — is what makes the format portable.

```
on_pickup:
  - read: zones 1-6
  - parse: validation_card YAML
  - acquire: lock via transition-status.sh      # L1

per_iteration:
  - execute: implementation
  - run: all success_criteria as bash
  - emit: pass | fail | retry_with_reason

on_terminal_state:
  pass:          status → done; archive
  budget_exhausted: status → parked          # L2 — with context, never silence
```

| CONSUMER | HOW IT READS THE SPEC |
|-------------------------|---|
| Claude Code | native — markdown + YAML + bash |
| Codex CLI | same file, cross-vendor |
| Kimi | same file — the sprinter class, atomic cranks |
| Gemini / Cursor / Aider | same file |
| anthive | dispatches one session per Task-Spec |
| taskship | runs a 3-phase mini-SDD against each spec |
| A human with a terminal | reads it as a checklist; runs the evals by hand |

THE BLACK-BOX GUARANTEE

Size → engine routing is **advisory** — a dispatcher heuristic keyed off `effort` , never an override of an author's explicit `execution_backend` . The agent contract keeps the executor a black box: the format specifies *what must be true*, never *how the engine must think*.

What shipped, and when

MAJOR IS THE SPEC FORMAT VERSION · MINOR IS ADDITIVE · **FORMAT_VERSION: 3** UNCHANGED SINCE 3.0

| VERSION | DATE | WHAT IT ADDED |
|---------|------------|--|
| 3.6.0 | 2026-07-24 | The L tier stops naming a vendor. Membership in <code>TS_LONG_HORIZON_BACKENDS</code> replaces the hard <code>glm</code> requirement; overridable per fleet. |
| 3.5.0 | 2026-07-23 | <code>tracker_ref</code> — the vendor-neutral tracker receipt written back by Register. <code>linear_ref</code> deprecated to an alias. |
| 3.4.0 | 2026-07-21 | The fork collapses. Six-tier model with two kinds; the <code>XL → SDD</code> escape removed; write-surface budgets; <code>children:</code> / <code>parent:</code> . |
| 3.3.0 | 2026-07-17 | Decomposition shape: the vertical-slice rule, the context-window ceiling, the breakdown quiz, expand–contract sequencing. |
| 3.2.0 | 2026-07-06 | Size-aware routing — effort becomes an engine-aware dispatch signal. |
| 3.1.0 | 2026-06-18 | <code>--gold-sanity</code> , <code>requires:</code> , the A2A <code>TaskState</code> dispatcher, DAG cycle detection, the published JSON Schema (Draft 2020-12), generic <code>backend_metadata</code> . |
| 3.0.0 | 2026-06-18 | Effort-scaled profiles, the Behavior zone + behavior↔eval traceability, <code>accept-task.sh</code> , conformance L0/L1/L2, open-string <code>execution_backend</code> . |
| 2.2 | 2026-06-02 | The key-optional HMAC sign-off envelope (<code>signed_off_sig</code>) — hand-stamping becomes detectable. |

VERSION DISCIPLINE

The canonical version string lives in `scripts/_lib.sh` and is duplicated in `SKILL.md` frontmatter, `plugin.json` , and `marketplace.json` . A doc-consistency lint (`lint-skill-docs.sh`) asserts all four match — the format holds itself to the same standard it imposes on tasks.

Anti-patterns — what people try that doesn't work

EACH ONE IS A REAL FAILURE MODE WITH A GATE BEHIND IT

| THE ATTEMPT | WHY IT FAILS |
|--|--|
| Editing <code>T-*.md</code> frontmatter directly | bypasses the lock and the ledger — use <code>transition-status.sh</code> |
| Skipping evals because "the task is simple" | every spec needs ≥ 3 evals; simple tasks have simple evals |
| A Context section over ~100 lines | you wrote a PRD — trim to what the executor needs |
| Vague guardrails ("be careful") | anti-patterns must name specific things not to do |
| Subjective evals ("looks good") | bash cannot check beauty — use an explicit human-checkpoint eval |
| Authoring XL/XXL as a runnable leaf | they are decomposition nodes — give them a <code>children:</code> block |
| Hand-writing <code>signed_off: true</code> | the HMAC envelope check rejects it — only the gate may stamp |
| Hand-writing <code>tracker_ref</code> | it is a receipt from Register, and tooling must tolerate <code>(none)</code> |

Where this sits in Converge

Task-Spec is **Pass 5 · Tasking** — the first pass below the barrier, and the cornerstone of the whole method. Pass 4's hardened plans decompose into these units; **Pass 6 Register** (opt-in) projects each signed spec onto one tracker issue; **Pass 7 Bind** binds a signed spec's exact hash to its runtime and emits the harness; **Pass 8 The Loop** drives one issue through a bounded attempttoverify cycle — the kernel enforces this spec's own `budget_iterations` and circuit breaker, and lands in one named terminal state — to a green-eval PR. Every one of those passes depends on the property this format provides: *the definition of done travels inside the file*.

THE HONEST BOUNDARY

A green eval proves the code matches the spec. It does not prove the spec was right. Task-Spec automates the *verification of the build*, not the *judgment of whether the build was worth making* — that judgment is what Converge's Phase 1 and its adversarial Consensus pass exist to protect.

COLOPHON

Specs that verify themselves don't need humans *in the middle of the loop.*

Task-Spec ships as a plugin at **v3.6.0** with a JSON Schema (Draft 2020-12), an L0/L1/L2 executor-conformance suite, 21 scripts, and dispatch recipes for Claude Code, Codex, Kimi, Cursor, Gemini, taskship, and anthive. This document was reconciled against `scripts/_lib.sh`, the changelog, the concept references, and the shipped test suites.

SKILL

`skills/task-spec` · Converge Pass 5

FORMAT

`format_version: 3` · additive since 3.0.0

LICENCE

MIT · built by Luan Moreno

Supersedes the v3.2.0 deep dive. What changed since: the fork to SDD was removed and the effort model became a six-tier recursive engine (3.4); the tracker receipt became vendor-neutral (3.5); and the L tier stopped naming a single vendor, making the portability claim true at the gate as well as in the prose (3.6).